

1. Terraform state file kya karti hai?

Terraform state ko simple language mein **Terraform ki memory** samjho.

Terraform code

↓

Terraform state

↓

AWS ke actual resources

Example:

```
resource "aws_s3_bucket" "claims" {  
    bucket = "insurance-claims-prod"  
}
```

State file mein mapping store hoti hai:

aws_s3_bucket.claims

↓

Actual AWS bucket: insurance-claims-prod

State ke bina Terraform ko accurately pata nahi chalega ki:

- Kaunsa AWS resource Terraform ne create kiya
- Resource ka actual ID/ARN kya hai
- Existing resource update karna hai ya naya banana hai
- Kaunsa resource delete ho chuka hai
- Infrastructure code aur AWS ke beech kya difference hai

2. Local state mein problem kya hai?

By default state locally create hoti hai:

terraform.tfstate

Agar Developer A ke laptop par state hai:

Developer A laptop

└── terraform.tfstate

To Developer B ke paas updated state nahi hogi.

Developer A state: Version 10

Developer B state: Version 8

Dono Terraform apply karenge toh resource duplication, conflicting updates ya state inconsistency ho sakti hai.

Isliye team environment mein state centrally store karte hain:

Developers / CI-CD pipeline

↓

S3 bucket

↓

terraform.tfstate

Sabhi developers aur CI/CD pipeline **same remote state** access karte hain.

Backend resources Terraform se kaise create hote hain?

Yahan ek **bootstrap problem** hoti hai:

State store karne ke liye S3 bucket chahiye

↓

S3 bucket Terraform se create karni hai

↓

Lekin Terraform state kahan rakhega?

Iska production solution:

Step 1: Separate bootstrap Terraform configuration

Step 2: S3 state bucket create

Step 3: Main project mein S3 backend configure

Step 4: Local state ko remote S3 mein migrate

Bootstrap folder

terraform/

├── bootstrap/

│ ├── main.tf

│ ├── providers.tf

│ └── outputs.tf

│

└── environments/

├── dev/

├── qa/

└── prod/

Bootstrap main.tf

```
resource "aws_s3_bucket" "terraform_state" {  
  bucket = "company-insurance-terraform-state"
```

```

lifecycle {
  prevent_destroy = true
}

tags = {
  Name = "Terraform Remote State"
  ManagedBy = "Terraform"
}
}

resource "aws_s3_bucket_versioning" "terraform_state" {
  bucket = aws_s3_bucket.terraform_state.id

  versioning_configuration {
    status = "Enabled"
  }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "terraform_state" {
  bucket = aws_s3_bucket.terraform_state.id

  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}

resource "aws_s3_bucket_public_access_block" "terraform_state" {
  bucket = aws_s3_bucket.terraform_state.id

  block_public_acls = true
  block_public_policy = true
  ignore_public_acls = true
  restrict_public_buckets = true
}

```

Important protections:

Versioning → Purani state recover kar sakte hain

Encryption → State encrypted at rest

Public access block → State internet par expose nahi hogi

prevent_destroy → Bucket accidentally destroy nahi hogi

IAM permissions → Authorized users/pipeline hi access kare

HashiCorp bhi state recovery ke liye backend S3 bucket par versioning enable karne ki strong recommendation deta hai.

[\[developer....hicorp.com\]](#)

6. Main project mein backend.tf

Bootstrap apply hone ke baad main insurance project mein:

```
terraform {  
  backend "s3" {  
    bucket = "company-insurance-terraform-state"  
    key = "insurance-claim/dev/terraform.tfstate"  
    region = "ap-south-1"  
    encrypt = true  
    use_lockfile = true  
  }  
}
```

Phir:

```
terraform init
```

Agar pehle local state exist karti hai aur usse S3 mein migrate karna hai:

```
terraform init -migrate-state
```

Flow:

Local terraform.tfstate

↓

```
terraform init -migrate-state
```

↓

S3 remote backend

↓

insurance-claim/dev/terraform.tfstate

Do developers ek saath code change karein toh kya hoga?

Yahan **Git conflict** aur **Terraform state lock** ko alag samjho.

Git code conflict

Developer A aur Developer B same Terraform file edit karte hain:

Developer A: EC2 instance type change

Developer B: EC2 security group change

Isko Git branches aur pull requests manage karte hain:

Developer branch

↓

Pull Request

↓

Terraform validate

↓

Security scan

↓

Terraform plan

↓

Review and approval

↓

Merge

State write conflict

Agar dono simultaneously terraform apply chala dein, state locking manage karegi:

Developer A

terraform apply

↓

Lock acquired

↓

AWS resource update starts

Developer B

terraform apply

↓

Lock acquire nahi hua

↓

Operation blocked

Possible error roughly:

Error acquiring the state lock

Lock Info:

ID: ...

Operation: OperationTypeApply

Who: developer-a

Developer B ko pehle Developer A ka apply complete hone dena chahiye. Apply complete hote hi lock normally automatically release ho jata hai.

Kya lock Developer B ke code ko merge hone se rokta hai?

Nahi.

Lock sirf Terraform state ko concurrent write se protect karta hai.

Git branch/PR

↓

Code changes manage karta hai

Terraform state lock

↓

Infrastructure apply execution manage karta hai

Agar Developer A aur Developer B code changes merge kar dete hain, phir final merged code se pipeline terraform plan aur terraform apply chalati hai.

Production mein ideal flow:

Developer A branch —┐

└── Pull Request checks

Developer B branch └┐

↓

Main branch merge

↓

Single deployment pipeline

↓

Terraform plan/apply

↓

Remote state lock

Best practice yeh hai ki har developer directly production par apply na chalaye. **CI/CD pipeline ko production apply ka primary owner** rakha jata hai.

DEV, QA aur PROD state kaise maintain hogi?

Har environment ki separate state honi chahiye:

S3 bucket

└── insurance-claim/dev/terraform.tfstate

└── insurance-claim/qa/terraform.tfstate

└── insurance-claim/prod/terraform.tfstate

Isse:

DEV change → DEV state

QA change → QA state

PROD change → PROD state

DEV ka apply PROD state ko modify nahi karega.

Production-grade multi-account setup mein stronger isolation ke liye alag AWS accounts bhi ho sakte hain:

DEV AWS Account

└── DEV state

QA AWS Account

└── QA state

PROD AWS Account

└── PROD state

Interview-ready answer

I store Terraform state in a secure and versioned S3 remote backend so that developers and CI/CD pipelines use the same centralized state. Earlier, DynamoDB was commonly used for distributed state locking through a table containing a string partition key named LockID. However, DynamoDB-based locking is now deprecated in Terraform's S3 backend, so for new implementations I use S3 native locking with `use_lockfile = true`.

When one Terraform operation is modifying infrastructure, it acquires the lock. Any second developer or pipeline attempting a conflicting operation cannot acquire the lock and Terraform stops that operation, preventing concurrent state writes. I also maintain separate state keys for DEV, QA, and PROD and allow production apply only through an approved CI/CD pipeline.

Bilkul bhai. Confusion isliye ho raha hai kyunki yahan **do alag Terraform executions** hote hain: pehla remote-state S3 bucket banane ke liye, aur doosra actual AWS infrastructure banane ke liye. Main exact sequence scratch se samjhata hoon.

Complete Terraform state flow

Step 1: Terraform code likha

Step 2: terraform init

Step 3: terraform plan

Step 4: terraform apply

Step 5: Local terraform.tfstate bani

Step 6: S3 backend configure kiya

Step 7: Local state S3 mein migrate ki

Step 8: Future state directly S3 mein update hoti rahegi

Phase 1: Terraform se S3 state bucket banana

Sabse pehle humein ek S3 bucket chahiye jisme baad mein Terraform state rakhenge.

Lekin initial problem ye hai:

Terraform state store karne ke liye S3 bucket chahiye

S3 bucket abhi exist nahi karti

S3 bucket bhi Terraform se create karni hai

Isko **Terraform bootstrap process** kehte hain.

Step 1: Bootstrap folder banate hain

insurance-claim-devsecops/

└── terraform/

└── bootstrap/

| └── providers.tf

| └── main.tf

| └── outputs.tf

|

└── infrastructure/

└── backend.tf

└── providers.tf

└── main.tf

└── variables.tf

Difference:

bootstrap/

└── Sirf Terraform state bucket create karega

infrastructure/

└── Actual application infrastructure create karega

Application infrastructure mein VPC, Lambda, API Gateway, DynamoDB, ECS, RDS jaise resources aa sakte hain.

Step 2: Bootstrap provider configuration

terraform/bootstrap/providers.tf:

```
terraform {
```

```
  required_version = ">= 1.10.0"
```

```
  required_providers {
```

```
    aws = {
```

```
      source = "hashicorp/aws"
```

```
version = "~> 6.0"
```

```
}
```

```
}
```

```
}
```

```
provider "aws" {
```

```
region = "ap-south-1"
```

```
}
```

Is file ka meaning:

- Terraform ka supported version define kiya
- AWS provider download karne ko bola
- AWS Mumbai region select kiya

Step 3: S3 state bucket ka Terraform code

terraform/bootstrap/main.tf:

```
resource "aws_s3_bucket" "terraform_state" {
```

```
bucket = "prashant-insurance-terraform-state"
```

```
lifecycle {
```

```
prevent_destroy = true
```

```
}
```

```
tags = {
```

```
Name = "Insurance Terraform State"
```

Environment = "Shared"

ManagedBy = "Terraform"

}

}

resource "aws_s3_bucket_versioning" "terraform_state" {

bucket = aws_s3_bucket.terraform_state.id

versioning_configuration {

status = "Enabled"

}

}

resource "aws_s3_bucket_server_side_encryption_configuration" "terraform_state" {

bucket = aws_s3_bucket.terraform_state.id

rule {

apply_server_side_encryption_by_default {

sse_algorithm = "AES256"

}

}

}

```
resource "aws_s3_bucket_public_access_block" "terraform_state" {
```

```
  bucket = aws_s3_bucket.terraform_state.id
```

```
  block_public_acls = true
```

```
  block_public_policy = true
```

```
  ignore_public_acls = true
```

```
  restrict_public_buckets = true
```

```
}
```

Is code se:

S3 bucket create hogi

Versioning enable hogi

Encryption enable hogi

Public access block hoga

Accidental Terraform destroy se bucket protected hogi

HashiCorp S3 state bucket par versioning enable karne ki strong recommendation deta hai, jisse accidental deletion ya human error ke case mein previous state recover ki ja sake. [\[developer....hicorp.com\]](#)

Bucket name globally unique hona chahiye. Upar wala naam example hai, available na ho toh unique suffix lagana padega.

Step 4: Bootstrap Terraform initialize karna

Bootstrap folder mein jaate hain:

```
cd terraform/bootstrap
```

```
terraform init
```

terraform init kya karega?

AWS provider download karega

Current folder initialize karega

Backend configuration check karega

Abhi humne remote backend configure nahi kiya hai, isliye bootstrap configuration default **local backend** use karegi.

Is stage par state file abhi create nahi hoti. `terraform init` primarily working directory aur providers initialize karta hai.

Step 5: Terraform plan generate karna

`terraform plan`

Terraform batayega:

Plan: 4 to add, 0 to change, 0 to destroy

Ismein likely ye resources honge:

S3 bucket

S3 bucket versioning configuration

S3 encryption configuration

S3 public access block

`terraform plan` normally infrastructure create nahi karta. Ye proposed changes calculate karke dikhata hai.

Step 6: Terraform apply chalana

`terraform apply`

Approval ke liye Terraform puchega:

Do you want to perform these actions?

Enter a value:

Enter karo:

yes

Ab Terraform:

1. AWS APIs call karega
2. S3 bucket create karega
3. Versioning configure karega
4. Encryption configure karega
5. Public access block configure karega
6. Created resources ki details local state mein save karega

Step 7: Pehli local state file create hogi

Apply successful hone ke baad bootstrap folder mein ye file aayegi:

terraform/bootstrap/

├── main.tf

├── providers.tf

├── terraform.tfstate

└── terraform.tfstate.backup

Main state file:

terraform.tfstate

Ismein mapping hogi:

Terraform resource:

aws_s3_bucket.terraform_state

Actual AWS resource:

prashant-insurance-terraform-state

Yaani pehli local state **S3 bucket create karne wale bootstrap infrastructure ki state** hai.

Important:

S3 bucket Terraform se create ho gayi

Lekin bootstrap ki state abhi local computer/CloudShell mein hai

Phase 2: Actual infrastructure ki state S3 mein rakhna

Ab S3 state bucket AWS mein available hai. Ab hum actual infrastructure configuration ko bolenge:

Apni state local file mein nahi, is S3 bucket mein store karo.

Step 8: Main infrastructure mein backend file banana

```
terraform/infrastructure/backend.tf:
```

```
terraform {
```

```
  backend "s3" {
```

```
    bucket = "prashant-insurance-terraform-state"
```

```
    key = "insurance-claim/dev/terraform.tfstate"
```

```
    region = "ap-south-1"
```

```
    encrypt = true
```

```
    use_lockfile = true
```

```
  }
```

```
}
```

Property meanings:

bucket

└── State kis S3 bucket mein store hogi

key

└── Bucket ke andar state object ka path

region

└── S3 bucket kis AWS region mein hai

encrypt

└── State ko encrypted form mein store karna

use_lockfile

└── Multiple Terraform operations ko same state par

simultaneously write karne se rokna

HashiCorp ke current S3 backend mein `use_lockfile = true` native S3 state locking enable karta hai. DynamoDB-based locking deprecated hai. [\[developer....hicorp.com\]](https://developer.hashicorp.com/terraform/language/backends)

S3 ke andar object path hoga:

s3://prashant-insurance-terraform-state/

└── insurance-claim/

└── dev/

└── terraform.tfstate

Locking ke waqt ek lock object bhi temporarily use ho sakta hai:

insurance-claim/dev/terraform.tfstate.tflock

Step 9: Actual infrastructure code likhna

For example, `terraform/infrastructure/main.tf`:


```
resource "aws_s3_bucket" "claim_documents" {
```

```
  bucket = "prashant-insurance-claim-documents"
```

```
  tags = {
```

```
    Name = "Insurance Claim Documents"
```

```
    Environment = "DEV"
```

```
    ManagedBy = "Terraform"
```

```
  }
```

```
}
```

Ye actual application bucket hai.

Do buckets ko mix mat karna:

prashant-insurance-terraform-state

└── Terraform state store karegi

prashant-insurance-claim-documents

└── Insurance application documents store karegi

Step 10: Main infrastructure initialize karna

Ab infrastructure folder mein jaate hain:

```
cd ../infrastructure
```

```
terraform init
```

Terraform backend configuration read karega:

Backend type: S3

Bucket: prashant-insurance-terraform-state

Key: insurance-claim/dev/terraform.tfstate

Terraform successful message dega:

Successfully configured the backend "s3"!

Abhi state object S3 mein empty ya absent ho sakta hai, kyunki actual infrastructure ka apply abhi nahi hua.

Step 11: Main infrastructure plan

terraform plan

Terraform:

1. S3 remote backend check karega
2. Current remote state read karega
3. Terraform code read karega
4. AWS infrastructure check karega
5. Required changes ka plan banayega

Pehli baar output aa sakta hai:

Plan: 1 to add, 0 to change, 0 to destroy

Step 12: Main infrastructure apply

terraform apply

yes enter karne ke baad:

Terraform AWS resource create karega

↓

Resource ki ID aur details collect karega

↓

Remote state prepare karega

↓

State directly S3 backend mein save karega

Ab actual application state S3 mein hogi:

s3://prashant-insurance-terraform-state/

└── insurance-claim/dev/terraform.tfstate

Is folder mein normal terraform.tfstate file create nahi hogi, kyunki backend remote S3 hai.

Pura process ek diagram mein

BOOTSTRAP PHASE

bootstrap/main.tf

↓

terraform init

↓

terraform plan

↓

terraform apply

↓

AWS mein state bucket create

↓

Bootstrap ki local terraform.tfstate create

MAIN INFRASTRUCTURE PHASE

infrastructure/backend.tf

↓

S3 remote backend configure

↓

terraform init

↓

terraform plan

↓

terraform apply

↓

AWS infrastructure create

↓

Application state S3 mein store

Agar actual infrastructure pehle se local state ke saath bana hua ho

Suppose humne pehle infrastructure create kar diya:

infrastructure/
└── terraform.tfstate

Aur baad mein S3 backend add kiya.

Tab normal terraform `init` ke bajaye migration command chalte hain:

```
terraform init -migrate-state
```

Terraform local state ko remote S3 backend mein copy/migrate karega:

Local terraform.tfstate

↓

```
terraform init -migrate-state
```

↓

S3 remote backend

↓

insurance-claim/dev/terraform.tfstate

State migration ke baad verify karna chahiye:

```
terraform state list
```

Ye remote state mein managed resources dikhayega.

Multiple developers mein kya hota hai?

Assume Developer A aur Developer B same repository clone karte hain.

Dono ke backend.tf mein same configuration hai:

```
backend "s3" {  
  bucket = "prashant-insurance-terraform-state"  
  key = "insurance-claim/dev/terraform.tfstate"  
  region = "ap-south-1"  
  encrypt = true  
  use_lockfile = true  
}
```

Developer A apply chalata hai

Developer A

↓

```
terraform apply
```

↓

S3 lock acquire

↓

Infrastructure update

↓
S3 state update
↓
Lock release

Isi waqt Developer B apply chalata hai

Developer B
↓
terraform apply
↓
Lock acquire karne ki attempt
↓
Lock already Developer A ke paas
↓
Developer B ka operation stop

Terraform supported backend par state-writing operations ke liye locking automatically perform karta hai. Lock acquire na hone par Terraform operation continue nahi karta. [\[developer....hicorp.com\]](#)

Developer A ka operation complete hone ke baad Developer B ko latest code pull karke dobara plan chalana chahiye:

git pull
terraform plan

Production best practice

Developers normally apni branches par code change karte hain:

Developer A branch └─
├── Pull Request
Developer B branch └─
↓
Validation and review
↓
Main branch
↓
CI/CD pipeline plan
↓
Approval
↓
CI/CD pipeline apply
↓
S3 remote state and lock

Production mein individual developers ko directly terraform apply access dene ki jagah controlled CI/CD pipeline se apply karna safer hota hai.

DEV, QA aur PROD ka state separation

Same state file sab environments ke liye use nahi karenge.

S3 state bucket

- └── insurance-claim/dev/terraform.tfstate
- └── insurance-claim/qa/terraform.tfstate
- └── insurance-claim/prod/terraform.tfstate

DEV backend:

key = "insurance-claim/dev/terraform.tfstate"

QA backend:

key = "insurance-claim/qa/terraform.tfstate"

PROD backend:

key = "insurance-claim/prod/terraform.tfstate"

Isse DEV apply galti se PROD state modify nahi karega.

Sabse important distinction

Bootstrap state

- └── State S3 bucket ko track karti hai

Application state

- └── VPC, Lambda, ECS, RDS aur application resources track karti hai

Basic learning mein bootstrap state local reh sakti hai, lekin production-grade implementation mein bootstrap state ko bhi separately secure remote backend ya dedicated infrastructure-management process mein maintain kiya jata hai.

Short version yaad rakhne ke liye

1. Bootstrap Terraform local backend use karta hai

2. Bootstrap apply se S3 state bucket banti hai
3. Pehli bootstrap terraform.tfstate locally banti hai
4. Main project ke backend.tf mein S3 bucket configure hoti hai
5. Main project mein terraform init chalta hai
6. Main infrastructure ka apply hota hai
7. Main infrastructure ki state directly S3 mein store hoti hai
8. S3 lock concurrent apply ko prevent karta hai